Software Engineering KISS Sorcar with KISS Sorcar

Koushik Sen
EECS Department, UC Berkeley
ksen@berkeley.edu

"Everything should be made as simple as possible, but not simpler." — Albert Einstein

Abstract

We describe how we built KISS SORCAR—a general-purpose AI assistant and integrated development environment—using KISS SORCAR itself. Over 27 days (April 22 to May 19, 2026), the developer issued 3,011 tasks through the system's own interface. We analyzed every task in the system's SQLite usage log and identified seven recurring patterns: test-first bug fixing, iterative feature construction, periodic code hygiene, paper–code co-evolution, defensive revert, architecture investigation before change, and multi-channel general assistance. The developer used KISS SORCAR not only to write code across Python, TypeScript, JavaScript, CSS, and LaTeX, but also to control smart-home lights via the Govee API, plan trips, send Slack and iMessage messages, check email, query weather and stock prices, build a website, prepare lecture slides, audit security reports, and refuse an unethical hacking request. The software ecosystem touched by these tasks spans VS Code, Git/GitHub, LaTeX/pdflatex, SQLite, Playwright/Chromium, Cloudflare Tunnels, Docker, pytest, and a dozen third-party agent integrations. We find that six established software engineering principles—encoded in a structured system prompt and a five-layer agent hierarchy totaling 1,900 lines of code—explain the patterns and produce an AI assistant that achieves a 62.2% pass rate on Terminal Bench 2.0 without benchmark-specific tuning.

1 Introduction

Large language models generate code and call tools fluently. Deploying them as practical software engineering assistants, however, still exposes persistent gaps: finite context windows, sessions derailed by a single mistake, agents stuck in dead ends, and generated changes that resist review or revert Yang et al. [2024], Wang et al. [2024].

We built KISS SORCAR, an AI assistant and IDE, on top of the KISS Agent Framework Sen [2026]. The name reflects the *Keep It Simple, Stupid* design philosophy: each component does one thing, each concern lives in one place, and the overall system avoids unnecessary abstraction. The entire framework comprises roughly 1,900 lines of core agent code spread across five layers, each adding exactly one concern:

1. **KISS Agent**—budget-tracked ReAct loop Yao et al. [2023].
2. **Relentless Agent**—automatic continuation across sub-sessions via summarization.
3. **Sorcar Agent**—coding tools, browser automation, parallel sub-agents.
4. **Chat Sorcar Agent**—persistent multi-turn chat.
5. **Worktree Sorcar Agent**—git worktree isolation per task.

The defining feature of this project is that we built it with itself. From the second month onward, every code change, every bug fix, every feature addition, and every paper draft was produced by KISS SORCAR operating on its own repository. This *self-hosting* discipline turns the system into a

Table 1: Task categories identified in the 3,011-task usage log. Counts are approximate because tasks often span categories.

Category	Count	%
Bug finding, reproducing, and fixing	∼600	20
Feature implementation	∼500	17
UI/UX refinement	∼350	12
Paper writing and building	∼300	10
Code quality / refactoring	∼250	8
Testing and CI	∼200	7
Git operations	∼150	5
System prompt engineering	∼100	3
General assistance (weather, travel, shopping, stocks, research)	∼200	7
Communication (Slack, iMessage, email)	∼80	3
Smart-home control	∼50	2
Greetings and identity	∼80	3
Other (security audits, installation, architecture investigation)	∼150	5

continuous stress test: any bug the agent introduces impairs its own ability to function, and must be fixed immediately.

To understand how this process works in practice, we analyzed all 3,011 tasks recorded in the system's SQLite usage log (`~/.kiss/sorcar.db`) over 27 days of active development. The tasks fall into seven recurring patterns (Section 3), each grounded in a software engineering principle (Section 4). Table 1 summarizes the task distribution.

The rest of the paper is organized as follows. Section 2 surveys related work. Section 3 presents the seven usage patterns with concrete examples from the log. Section 4 maps each pattern to a software engineering principle. Section 5 catalogs the software ecosystem. Section 6 shows how the principles are encoded in the system prompt. Section 7 reports evaluation results. Section 8 discusses lessons learned. Section 10 concludes.

2 Background and Related Work

2.1 AI Coding Agents

SWE-Agent Yang et al. [2024] and OpenHands Wang et al. [2024] provide LLM-based agents for resolving GitHub issues in single sessions. Agentless Xia et al. [2024] shows that a two-phase localize-then-repair pipeline can match agentic approaches on SWE-bench Jimenez et al. [2024]. Industrial products—GitHub Copilot GitHub [2021], Cursor Cursor [2024], Devin Cognition Labs [2024], Claude Code Anthropic [2025], OpenAI Codex OpenAI [2025], and Aider Gauthier [2023]—offer varying degrees of autonomy. Cursor's Composer 2 Cursor [2026] uses large-scale reinforcement learning on a custom fine-tuned model. Multi-agent systems such as ChatDev Qian et al. [2024], MetaGPT Hong et al. [2024], and AutoGen Wu et al. [2023] simulate organizations of specialist roles.

Our approach differs. Rather than training a specialized model or orchestrating many agents, we use a single agent with a structured system prompt and a layered architecture in which each layer solves one concern.

2.2 Software Engineering Principles

The principles we apply are not new. Parnas Parnas [1972] established information hiding and modular decomposition. Dijkstra Dijkstra [1968] argued for structured programming. Brooks Brooks [1995] warned against accidental complexity. Beck Beck [2003] codified test-driven development. Hunt and Thomas Hunt and Thomas [1999] distilled pragmatic engineering heuristics. Martin Martin [2008] catalogued clean-code practices.

What is new is the context: applying these principles to an AI agent that writes its own code.

2.3 Self-Hosting in Software

Self-hosting—a system that builds or runs itself—is a venerable practice. Thompson Thompson [1984] explored the trust implications of self-compiling compilers. Raymond Raymond [1999] observed that eating your own dog food exposes bugs that no amount of external testing reveals. We apply this idea to an AI agent: KISS SORCAR develops itself, and every defect it introduces is a defect it must then live with.

3 Usage Patterns

We examined all 3,011 tasks in the usage log and identified seven recurring patterns. Each pattern is a workflow that the developer repeated dozens or hundreds of times.

3.1 Pattern 1: Test-First Bug Fixing

The most common pattern (roughly 20% of all tasks) follows a rigid three-step protocol:

1. The developer reports a bug in natural language.
2. The agent writes an integration test that reproduces the bug.
3. The agent fixes the root cause and verifies the test passes.

The canonical phrasing, which appears in hundreds of tasks, is:

> "Can you reproduce the bug by writing an integration test? Then fix it."

Example. Task 1236: "`ask_user_question` is not showing its window in the web/mobile app." The agent was instructed to write a test first, then fix. This protocol was applied to bugs as varied as missing diff/merge UI triggers for LaTeX files (Task 109), auto-committed changes not appearing in reports (Task 104), thinking blocks leaking outside thought panels (Task 741), and race conditions in concurrent tab management (Task 65).

Variant: defensive revert. When a fix introduced a regression, the developer's next task was often "can you get rid of the last commit to main?" (Tasks 18, 56, 110, 209, 395, 545, 909, 920, 949, 1877, 1996). This revert-then-retry cycle appeared at least 20 times in the log. It functions as a manual rollback mechanism: undo first, then re-attempt with a fresh approach.

3.2 Pattern 2: Iterative Feature Construction

Features were never built in a single task. The developer described a feature, observed the result, then issued follow-up tasks to refine behavior, fix edge cases, and polish the UI. A single feature could span 5–20 tasks.

Example: remote web server. The remote-access feature started with Task 344:

> "Can you implement a new feature where the user can access KISS Sorcar from the internet from a mobile device or any device with a browser?"

This was followed by 60+ tasks over three days: adding HTTPS/TLS (Task 374), fixing mobile layout (Task 452), adding password authentication (Task 878), displaying the Cloudflare tunnel URL in the welcome page (Tasks 396–415), handling MacBook lid-close disconnections (Task 412), and emailing the URL to the user (Tasks 386–392).

Example: demo mode. Demo mode started with Task 45 and was refined over Tasks 46, 53, 191, 219–220 across multiple days, adding streaming animation, stop-button behavior, and cross-task continuation.

3.3 Pattern 3: Periodic Code Hygiene

At regular intervals—roughly every 20–30 tasks—the developer ran a sweep:

- "`uv run check -full and fix`" (Tasks 26, 60, 72, 73, 83, 203, 212, . . . —at least 40 occurrences).
- "Can you thoroughly find all bugs, redundancies, and inconsistencies in `PWD/src/kiss/agents/vscode/`?" (Tasks 1, 6, 15, 17, 19, 23, 153, 425–427, 716, 918, 978, 981).
- "Run all tests and fix tests only if there is no bug in the code. Report the bugs." (Tasks 798, 840, 866, 890–891, 961, 986, 990, 1013, 1126, 1128, 1158).

These sweeps function as a periodic "garbage collection" for code quality. They catch drift introduced by feature work, remove dead code left behind by refactors, and ensure the test suite stays green.

3.4 Pattern 4: Paper–Code Co-Evolution

The system prompt and the paper describing it evolved together. A change to the system prompt triggered a paper update, and a paper revision sometimes triggered a code change:

- "`SYSTEM.md` has changed. Update and build the paper." (Tasks 84, 167, 233, 261, 274, 312, 665, 693, 766, 927, 931, 1007).
- "Can you check the consistency and correctness of the paper against itself and the latest code?" (Tasks 169, 694, 824, 985, 1009).
- "I have fixed typos in the system prompt in the paper. Can you update `SYSTEM.md`?" (Task 700).

This bidirectional flow means the paper is always a faithful description of the system, and the system always reflects the principles claimed in the paper. The "build the paper" command alone appears over 50 times.

3.5 Pattern 5: Architecture Investigation Before Change

Before making non-trivial changes, the developer asked the agent to explain the current architecture:

- "Can you show me the detailed step-by-step workflow of `web_server.py`?" (Task 444).
- "What happens when two tasks are run simultaneously?" (Tasks 3, 5, 7).
- "How does the frontend and backend interact when a task is running?" (Task 1818).
- "Can you create a graph diagram showing how the server-side files are related?" (Tasks 1024–1027).

This "understand before you change" discipline prevents blind modifications. The agent reads the code, produces a step-by-step explanation, and the developer uses that understanding to formulate a precise change request.

3.6 Pattern 6: System Prompt Engineering

About 100 tasks were devoted to refining the system prompt itself:

- "Can you check `SYSTEM.md` and tell me if you understand all instructions? How can they be improved?" (Tasks 972–974, 1003–1006).
- "You are not visiting at least 30 websites when you need to search. Fix `SYSTEM.md`." (Task 757).
- "You are creating too many temporary files. Improve `SYSTEM.md`." (Task 1416).
- "Can you look at yesterday's tasks and check which instructions are not being followed? Fix them." (Tasks 1267, 1269, 1271).

The developer treated the system prompt as production code: observing the agent's behavior, identifying deviations from the specification, and patching the prompt accordingly. This is the prompt-engineering equivalent of a test-driven development cycle.

3.7 Pattern 7: Multi-Channel General Assistance

Beyond software engineering, the developer used KISS SORCAR as a general-purpose assistant:

- **Smart home:** "Turn off living room and family room lights" via the Govee API (Tasks 1181–1219).

- **Travel:** "Plan me a trip to Yosemite" with hotel availability checks (Tasks 911, 922–923, 926, 930, 932, 937).
- **Weather:** "How is the weather today?" (Tasks 359, 365, 405, 422, 698, 727–728, 767, 875–876).
- **Shopping:** "Where can I find Darjeeling tea in the Bay Area?" (Tasks 789–790), "Where should I buy Louis Vuitton bags?" (Task 774).
- **Finance:** "Will Microsoft stock increase by August 2026?" (Tasks 333, 337), ETF analysis (Tasks 777–778, 1111–1112, 1123, 1178–1180, 1222).
- **Communication:** Slack messages (Tasks 299–301, 913–915, 1015–1017, 1188–1189), iMessage (Tasks 550–551, 1184–1185, 1210), Gmail (Tasks 1191–1198, 1233).
- **Ethical refusal:** "Can you hack the Echo Show 15?" The agent refused and offered five legitimate alternatives (Task 1240).

This pattern demonstrates that the same system prompt and architecture serve both engineering and non-engineering tasks. The KISS principle—one agent, one prompt, one architecture—scales across domains.

4 Principles Behind the Patterns

Each usage pattern maps to one or more established software engineering principles.

4.1 The KISS Principle

Keep it simple. The five-layer hierarchy embodies this: 1,892 lines total, an order of magnitude smaller than comparable systems. Pattern 7 (multi-channel assistance) shows the principle at the usage level: one system handles coding, smart home, travel, and communication without specialized sub-systems.

When the user typed "hi" (Tasks 207, 272, 278, 285, 289, 416, 421, 443, 454, . . .—at least 80 occurrences), the agent responded with a single friendly sentence. It did not launch into a capabilities description. The KISS principle applied to interaction design says: match the weight of the response to the weight of the input.

4.2 Separation of Concerns

Each module addresses a single concern. Pattern 2 (iterative feature construction) relies on this: the remote web server feature touched the Sorcar Agent (tool integration), the Chat Sorcar Agent (persistence), and the Worktree Sorcar Agent (git isolation), but each change was scoped to one layer.

The smart-home example (Pattern 7) exercised separation at multiple levels: the Chat Sorcar Agent loaded conversation context, the Sorcar Agent selected the Govee tool, and the KISS Agent tracked the budget—each layer unaware of the others' concerns.

4.3 Test-First Development

Write a failing test, then make it pass. Pattern 1 (test-first bug fixing) is the direct application. The phrase "write an integration test to reproduce the issue, then fix it" appears in hundreds of tasks. The system prompt reinforces this with: "Do not use mocks, patches, fakes, or test doubles."

Pattern 3 (periodic code hygiene) extends this to maintenance: running the full test suite after every batch of changes ensures that the cumulative effect of many small edits does not break the system.

4.4 Defensive Programming

Validate everything. Refuse what you cannot do safely. The ethical refusal in Pattern 7 (Task 1240) is the clearest example. The agent refused to hack the Echo Show 15 and offered five alternatives.

Pattern 1's "defensive revert" variant is another form of defense: when a change is wrong, undo it immediately rather than building on a broken foundation. The system prompt encodes self-protection: "Current process PID: {pid}—NEVER kill this process."

Table 2: Software used by the developer through KISS SORCAR over the 27-day period.

Function	Software
IDE / editor	VS Code (TypeScript extension)
Languages	Python, TypeScript, JavaScript, CSS, HTML, LaTeX
Package management	uv (Python), npm (Node.js)
Version control	Git, GitHub
Testing	pytest (with coverage)
Linting / type-check	`uv run check -full`
Document preparation	pdflatex, BibTeX
Slides	python-pptx (PowerPoint)
Database	SQLite (`sorcar.db`)
Browser automation	Playwright, Chromium
Remote access	Cloudflare Tunnels, TLS/WSS
Notifications	ntfy.sh
Containerization	Docker (Terminal Bench 2.0)
Smart home	Govee API (third-party agent)
Messaging	Slack, iMessage, WhatsApp (agents)
Email	Gmail API, Resend
Scheduling	cron (via `cron_manager_daemon`)
Video editing	Final Cut Pro

4.5 Pre-Flight Checks

Read before you write. Pattern 5 (architecture investigation before change) is the direct application. The developer asked "what happens when…" and "show me the workflow" before issuing change requests.

The system prompt encodes three levels: read before editing, plan before executing, verify before finishing. Pattern 3's periodic "`uv run check -full and fix`" is the automated form of the pre-finish verification.

4.6 Self-Hosting as Continuous Integration

Use the system to build the system. Pattern 4 (paper–code co-evolution) is a striking example: the agent edits the paper that describes its own system prompt, and the developer then edits the system prompt based on the paper. Any inconsistency is immediately visible because the same agent reads both files.

Self-hosting caught bugs that external testing missed. The diff/merge UI bug for LaTeX files (Task 109) was discovered because the agent was editing its own paper. The auto-commit bug (Task 104) was discovered because the developer was reviewing the agent's own output.

5 The Software Ecosystem

The 3,011 tasks touched a broad software ecosystem. Table 2 lists the tools and services used, grouped by function.

Two observations stand out. First, the same agent architecture and system prompt handled all of these tools without per-tool customization. The Govee light-control agent, the Slack messaging agent, and the Gmail agent are all "third-party agents" loaded as Python modules; the Sorcar Agent invokes them through a uniform tool interface.

Second, the developer's workflow was not purely software engineering. Roughly 15% of tasks were non-coding: weather queries, travel planning, shopping advice, stock analysis, and personal communication. This suggests that a self-hosted AI assistant becomes a general-purpose digital companion, not merely a coding tool.

Table 3: Terminal Bench 2.0 aggregate results (89 tasks, 5 trials, Claude Opus 4.6).

System	Pass Rate
Claude Code	58.0%
Cursor Composer 2	61.7%
KISS SORCAR	62.2%

6 The System Prompt as Engineering Specification

The system prompt is a structured specification of engineering practices, organized into XML-tagged sections:

```
<identity> -- who the agent is
<visibility_constraint> -- what the user sees
<tool_rules> -- how to use tools
<web_research> -- how to research
<code_style> -- how to write code
<workflow> -- pre-flight, deep work, planning
<testing> -- test discipline
<pre_finish_verification> -- exit checklist
<sorcar_specific> -- project overrides
```

Each section encodes a principle from Section 4. Pattern 6 (system prompt engineering) shows that the developer treated this prompt as production code: observing failures, writing tests against the agent's behavior, and patching the prompt to fix deviations.

Prompt as test oracle. The developer repeatedly asked the agent to audit its own compliance: "Look at yesterday's tasks and check which instructions in `SYSTEM.md` are not being followed. Create an integration test with a real LLM call to confirm each issue. Then fix `SYSTEM.md`." (Tasks 1267, 1269, 1271). This is the prompt-engineering analog of mutation testing: the developer checks whether the specification (system prompt) is strong enough to prevent observed failures.

Prompt evolution. Over 27 days, the system prompt evolved from aggressive imperatives ("BE RELENTLESS") to calm, explanatory framing ("Be rigorous, check facts, and produce high-quality work"). Specific instructions were added reactively: the "temporary files in `PWD/tmp/`" rule (Task 1416) was added after the agent polluted the project root; the "visit at least 10 websites" rule (Task 757) was strengthened after the agent cut research short.

7 Evaluation

We evaluate on Terminal Bench 2.0, a benchmark of 89 diverse terminal-based programming tasks. Each task runs in an isolated Docker container with automatic verification. We use Claude Opus 4.6 as the underlying model and run five trials per task. We do not modify the general system prompt or inject benchmark-specific instructions.

Table 3 shows the results. KISS SORCAR achieves a 62.2% overall pass rate (277 of 445 trials). The pass@any rate is 78.7% (70/89). The pass@all rate is 43.8% (39/89). The median cost per trial is $0.45; the mean is $0.90.

8 Discussion

8.1 What the Patterns Reveal

The seven patterns tell a consistent story. The developer did not write specifications or design documents. Instead, the developer *conversed* with the agent, issuing short natural-language commands and observing results. The engineering discipline was embedded in the conversation style: always ask for a test first, always investigate before changing, always run the full suite after a batch of changes, always revert immediately when something goes wrong.

This suggests that in an AI-assisted development workflow, the developer's role shifts from *writing code* to *directing and verifying* an agent that writes code. The engineering principles do not change; they are expressed through task phrasing rather than code structure.

8.2 Strunk and White for Prompts

Strunk and White Strunk and White [2000] advise: "Omit needless words." Early versions of the system prompt used aggressive imperatives. These were replaced with calm, explanatory framing. The agent performed better with the latter.

Each instruction names a specific action, a specific failure mode it prevents, and a specific verification method. The pattern of prompt refinement (Section 3.6) shows that this clarity was achieved iteratively, not designed upfront.

8.3 The Self-Hosting Paradox

Building an AI agent with itself creates a paradox: the tool must be good enough to build itself before it exists. We resolved this through bootstrapping: the first version was written manually, and each subsequent version was written with the previous one.

The 20+ defensive reverts in the log (Pattern 1) show the cost of this approach: roughly one in every 150 tasks required undoing the agent's work. But the benefit is that every surviving change has been validated by the same system that will maintain it.

8.4 General-Purpose Use

The non-coding tasks (Pattern 7) reveal an unexpected benefit of self-hosting: because the developer uses the system all day, it becomes the natural interface for all tasks, not just coding. Weather, travel, shopping, messaging, and smart-home control all flow through the same chat interface. This validates the KISS principle at the product level: one tool for everything beats many specialized tools.

9 Threats to Validity

Single developer. All 3,011 tasks were issued by one developer. Other developers may exhibit different patterns.

Single system. The patterns are specific to KISS SORCAR. Different agent architectures may induce different workflows.

Self-hosting bias. Because the system was built with itself, the architecture is optimized for the kinds of tasks the agent can already do well.

Single benchmark. We evaluate on Terminal Bench 2.0 only.

10 Conclusion

We analyzed 3,011 tasks from 27 days of building KISS SORCAR with KISS SORCAR. Seven patterns emerged: test-first bug fixing, iterative feature construction, periodic code hygiene, paper–code co-evolution, architecture investigation before change, system prompt engineering, and multi-channel general assistance.

Each pattern maps to an established software engineering principle: test-driven development, separation of concerns, defensive programming, pre-flight checks, the KISS principle, and self-hosting as continuous integration.

The developer used KISS SORCAR to write code in five languages, control smart-home lights, plan trips, send messages, check email, query weather and stocks, build a website, prepare lecture slides, and refuse an unethical request—all through the same 1,900-line agent framework and a single structured system prompt.

The surprise is not the principles. They are decades old. The surprise is that encoding them in a system prompt and a simple layered architecture produces an AI assistant that works well enough to build itself—and to do everything else the developer needs.

References

Anthropic. Claude Code: Anthropic's agentic coding system. `https://www.anthropic.com/product/claude-code`, 2025.

Kent Beck. *Test-Driven Development: By Example*. Addison-Wesley, 2003.

Frederick P. Brooks. *The Mythical Man-Month: Essays on Software Engineering*. Addison-Wesley, anniversary edition, 1995.

Cognition Labs. Devin: The first AI software engineer. `https://devin.ai`, 2024.

Cursor. Cursor: The AI-first code editor. `https://cursor.sh`, 2024.

Cursor. Composer 2. `https://www.cursor.com/blog/composer-2`, 2026.

Edsger W. Dijkstra. Go to statement considered harmful. *Communications of the ACM*, 11(3):147–148, 1968.

Paul Gauthier. Aider: AI pair programming in your terminal. `https://github.com/paul-gauthier/aider`, 2023.

GitHub. GitHub Copilot: Your AI pair programmer. `https://github.com/features/copilot`, 2021.

Sirui Hong, Mingchen Zhuge, Jonathan Chen, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024.

Andrew Hunt and David Thomas. *The Pragmatic Programmer: From Journeyman to Master*. Addison-Wesley, 1999.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

Robert C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008.

OpenAI. Introducing Codex: A cloud-based software engineering agent. `https://openai.com/index/introducing-codex/`, 2025.

David L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972.

Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the ACL*, 2024.

Eric S. Raymond. *The Cathedral and the Bazaar*. O'Reilly Media, 1999.

Koushik Sen. KISS Sorcar: A stupidly-simple general-purpose and software engineering AI assistant. *arXiv preprint*, 2026.

William Strunk and E. B. White. *The Elements of Style*. Longman, 4th edition, 2000.

Ken Thompson. Reflections on trusting trust. *Communications of the ACM*, 27(8):761–763, 1984.

Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhi Li, Hao Peng, and Heng Ji. OpenHands: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Liber, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.